Calculator DevOps: An End-to-End CI/CD Pipeline for a Spring Boot REST API

CSE 816 Software Production Engineering, Mini-Project Report, M.Tech (CSE) Term I, 2026-27

Student	Ansh Gupta (IMT2023540), ansh.gupta@iiitb.ac.in
Faculty	Prof. B. Thangaraju
GitHub	https://github.com/Ansh-Gupta0310/calculator-devops
Docker Hub	https://hub.docker.com/r/anshgupta0310/calculator-devops

1. Overview

The project implements a Calculator REST API in Java 21 with Spring Boot 4.1.2 and Maven, and wraps it in a production-style pipeline built only with Git/GitHub, Jenkins, Docker and Docker Compose. The application is deliberately small; the engineering effort is in the automation around it. A push to **main** fires a GitHub webhook, Jenkins runs a six-stage declarative pipeline (checkout, build, test, image build, push, deploy), the new image is published to Docker Hub, the running Compose stack is updated in place, and the result is emailed to the author. In the verified run, **build #7**, all six stages passed in 1 min 6 s, all 19 tests passed, and the build was started by a GitHub push with no manual step (Appendix A).

2. Architecture

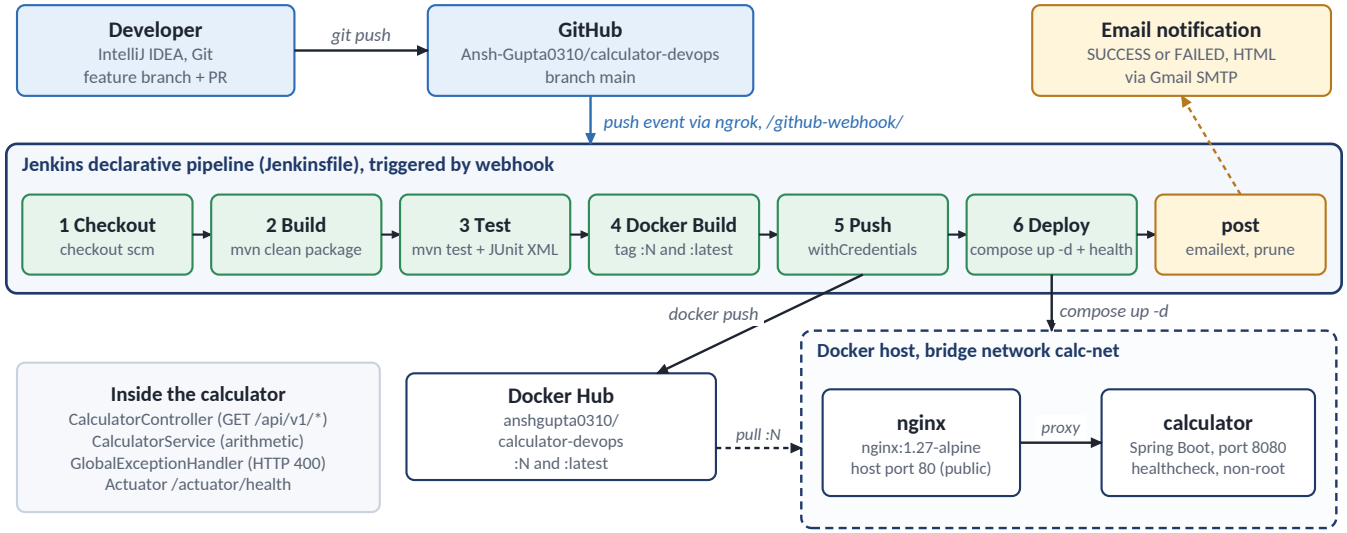


Figure 1. Pipeline and deployment architecture. Solid arrows are the delivery path; dashed arrows are the image pull and the email notification.

Development happens in IntelliJ IDEA on feature branches that are merged into **main** through pull requests. GitHub cannot call a Jenkins server on a laptop, so an **ngrok** tunnel gives Jenkins a public HTTPS URL that receives the webhook at `/github-webhook/`. Jenkins, the Docker engine and the deployed stack run on one Ubuntu host. The stack has two containers on the user-defined bridge network `calc-net`: Nginx is the only container that publishes a port (80) and proxies `/api/` and `/actuator/` to the calculator container, whose port 8080 is reachable only inside the network.

Inside the application, `CalculatorController` is a thin REST layer whose four GET endpoints under `/api/v1` (add, subtract, multiply, divide, each taking query parameters a and b) delegate to `CalculatorService`, which holds the arithmetic and throws `DivideByZeroException` when b is 0. A `@RestControllerAdvice` converts division by zero, missing parameters and non-numeric parameters into HTTP 400 with a structured body {timestamp, status, error, message, path}. Successful calls return {operation, a, b, result}. Spring Boot Actuator serves `/actuator/health` with liveness and readiness probes, which the Compose health check uses.

Test class	Type	Tests	What it verifies
`CalculatorServiceTest`	Unit, plain JUnit 5	10	every operation, negatives, decimals, multiply by zero, divide by zero throws
`CalculatorControllerTest`	Integration, @SpringBootTest + MockMvc	8	all four endpoints and their JSON; 400 for divide by zero, missing and non-numeric input; 404 for an unknown path
`CalculatorApplicationTests`	Context load	1	the Spring context starts with all beans wired

3. Pipeline stages

Stage	Implementation	Purpose and failure behaviour	#7
1. Checkout	`checkout scm`, `git log -1`	Fetches the exact commit that fired the webhook and logs it for traceability.	1 s
2. Build	`mvn -B -ntp clean package -DskipTests`	Compiles and packages the executable JAR. A compile error stops the pipeline. Tests are skipped here because stage 3 runs them.	6 s
3. Test	`mvn -B -ntp test`; `junit 'target/surefire-reports/*.xml'` in `post { always }`	Runs the 19 JUnit 5 tests and publishes the results even when a test fails, so the failure is visible in Jenkins. Any failure marks the build failed and no image is built or deployed.	9 s
4. Docker Build	`docker build -t IMAGE:${BUILD_NUMBER} -t IMAGE:latest .`	Builds the multi-stage image. The build-number tag ties every image to one Jenkins run.	3 s
5. Push	`withCredentials(dockerhub-creds)`, `docker login --password-stdin`, push both tags, `docker logout`	Publishes to Docker Hub with a token held in the Jenkins Credentials store. The token never appears in the repository and is masked in the log.	12 s
6. Deploy	`export DOCKER_IMAGE=IMAGE:${BUILD_NUMBER}`, `docker compose pull`, `docker compose up -d --remove-orphans`, `curl -fsS http://localhost/actuator/health`	Moves the stack to the exact image just pushed, then checks health through Nginx. A failed health call fails the build.	25 s

Trigger and notifications. The job uses "GitHub hook trigger for GITScm polling" with the Jenkinsfile loaded from SCM. The `post` block sends an HTML email through the Email Extension plugin on success and on failure (job, build number, image tag, link), using Gmail SMTP with an app password stored as a Jenkins credential, and `post { always }` prunes images older than 72 hours. Pipeline options add timestamps, a 20 minute timeout and a 15 build history limit. **Idempotency:** `docker compose up -d` recreates only services whose image or configuration changed, and each build pushes a unique tag. After build #7, `docker ps` showed the calculator container on tag :7 created 30 minutes earlier while `calc-nginx` had been up for 4 hours (Figure A4): re-running the pipeline replaced only the application container and never took the proxy down.

4. Containerization (Dockerfile)

- **Build stage** `maven:3.9.9-eclipse-temurin-21`: copies `pom.xml` and runs `mvn dependency:go-offline` before copying `src`, so the dependency layer is cached and a code-only change rebuilds quickly (3 s in build #7).
- **Runtime stage** `eclipse-temurin:21-jre-alpine`: receives only the JAR through `COPY --from=build --chown=app:app`; no JDK, Maven or source code ships in the image.
- **Security and runtime:** a non-root `app` user is created and selected with `USER app`; `EXPOSE 8080` documents the port; the exec-form `ENTRYPOINT ["java","-jar","/app/app.jar"]` makes the JVM PID 1 so it receives the stop signal.
- **Build context:** `.dockerignore` excludes `target/`, `.git/`, IDE folders, docs and secret files such as `.env` and `*.key`.

5. Deployment (Docker Compose)

- **calculator:** image `${DOCKER_IMAGE:-anshgupta0310/calculator-devops:latest}`; environment `SPRING_PROFILES_ACTIVE=prod` and `JAVA_TOOL_OPTIONS` (heap limits); `expose: 8080` (internal only); health check `wget -qO- http://localhost:8080/actuator/health` every 15 s with a 5 s timeout, 5 retries and a 20 s start period; `restart: unless-stopped`.
- **nginx:** `nginx:1.27-alpine`, `ports: 80:80`, `nginx.conf` mounted read-only, `depends_on: calculator: condition: service_healthy` so the proxy starts only after the API reports UP, `restart: unless-stopped`.
- **Network:** bridge network `calc-net`; Docker's embedded DNS resolves the service name `calculator` used in the Nginx upstream. A single `docker compose up -d` brings up the whole stack.

6. Version control practices

The history has 25 commits over two days, written progressively in Conventional Commits style (`feat:`, `test:`, `build:`, `ci:`). Five feature branches were merged through pull requests: #1 `feat/calculator-endpoints`, #2 `feat/dockerfile`, #3 `feat/docker-compose`, #4 `ci/jenkinsfile` and #5 `ci/email-notify`. The `.gitignore` excludes `target/`, IDE files, logs and secret files. The full history was scanned and contains no credentials; the Docker Hub token and the SMTP password exist only in the Jenkins Credentials store.

7. Challenges faced and solutions

Spring Boot 4 starters and test packages. IntelliJ's Spring Initializr generated a Spring Boot 4.1.2 project, where the web starter is `spring-boot-starter-webmvc` and each starter has its own `-test` companion. `@AutoConfigureMockMvc` also moved to `org.springframework.boot.webmvc.test.autoconfigure`, so Boot 3 examples did not compile. Importing from the Boot 4 package fixed the MockMvc integration tests.

Reaching a local Jenkins from GitHub. GitHub's webhook servers cannot call a Jenkins on localhost. An ngrok tunnel provided a public HTTPS address, which was set as the Jenkins URL; the webhook payload URL was pointed at `/github-webhook/` (the trailing slash is required) with content type `application/json`, and the job's GitHub hook trigger was enabled. Getting a push to start a build reliably took several iterations, visible as six "smoke-test webhook" commits; GitHub's Recent Deliveries page shows the HTTP response for every attempt and can redeliver a payload.

Email notifications. Gmail refuses the normal account password for SMTP, so an app password was generated and stored as a Jenkins credential. The first version of the `post` block did not deliver mail; giving `emailext` an explicit `to:` recipient and correcting the post conditions fixed it (commits 09ce004 and 3f2eded).

Running Docker from Jenkins without leaking secrets. The Docker Build, Push and Deploy stages run `docker` as the Jenkins user, which therefore needs access to the Docker daemon through the `docker` group. Docker Hub credentials are bound with `withCredentials` and passed through `--password-stdin`, so the token is masked in the console and never appears in process arguments or in Git.

Startup order and JVM settings. Spring Boot needs several seconds to start, so Nginx could proxy to an API that is not ready. The health check plus `depends_on: service_healthy` makes Nginx wait, and the Deploy stage verifies health through the proxy before the build is marked successful. The heap limits were first passed as `JAVA_OPTS`, which an exec-form `java -jar` entrypoint ignores; switching to `JAVA_TOOL_OPTIONS`, which the JVM reads by itself, made them take effect.

8. Results

- Build #7: started by a GitHub push, all six stages green, 1 min 6 s end to end (Figures A1, A6).
- 19 of 19 tests passed and were published by the Jenkins JUnit plugin (Figure A2).
- Image `anshgupta0310/calculator-devops:7` and `:latest` pushed to the public Docker Hub repository; success email received (Figure A3).
- Running stack: `calculator` healthy and `calc-nginx` published on port 80 (Figure A4). Through Nginx: 5 + 2.6 = 7.6, -5 - 2.6 = -7.6, -5 x 2.6 = -13.0, -5 / 2 = -2.5 (Figure A5); division by zero returns HTTP 400 with the message "Cannot divide by zero".

9. Requirement coverage

Rubric criterion	Marks	Where it is met
Spring Boot API + JUnit tests	2.0	four endpoints plus Actuator health; HTTP 400 with a structured body for invalid input and b = 0; 19 tests (10 unit, 8 MockMvc, 1 context load); sections 2 and 3
Git/GitHub practices	1.0	25 progressive commits, five feature branches merged by pull request, Java/Maven `.gitignore`, no secrets in history; section 6
Jenkins CI/CD pipeline	3.0	declarative Jenkinsfile with the six required stages, webhook trigger, JUnit publishing, Jenkins-managed Docker Hub credentials, email on success and failure, idempotent deploy; section 3
Dockerfile	1.5	multi-stage build, Alpine JRE runtime, non-root user, EXPOSE, cached dependency layer, `.dockerignore`, runs with one `docker run`; section 4
Docker Compose deployment	1.5	calculator plus Nginx proxy, network, Actuator health check, restart policy, environment variables, one `docker compose up`; section 5
Report, screenshots, demo	1.0	this report, Appendix A and the README; live demo: push a change and watch the next build deploy it

10. Limitations and future work

The Deploy stage waits a fixed 15 s before its health call; polling until healthy would be faster and more robust. Declarative pipelines run an implicit "Checkout SCM" before stage 1, which `options { skipDefaultCheckout() }` would remove. The webhook depends on the ngrok tunnel being up; a Jenkins host with a public address removes that dependency. The end-of-semester Capstone will extend this same pipeline with Ansible for configuration management, Kubernetes for orchestration, and Prometheus with Grafana for monitoring.

Appendix A. Screenshots

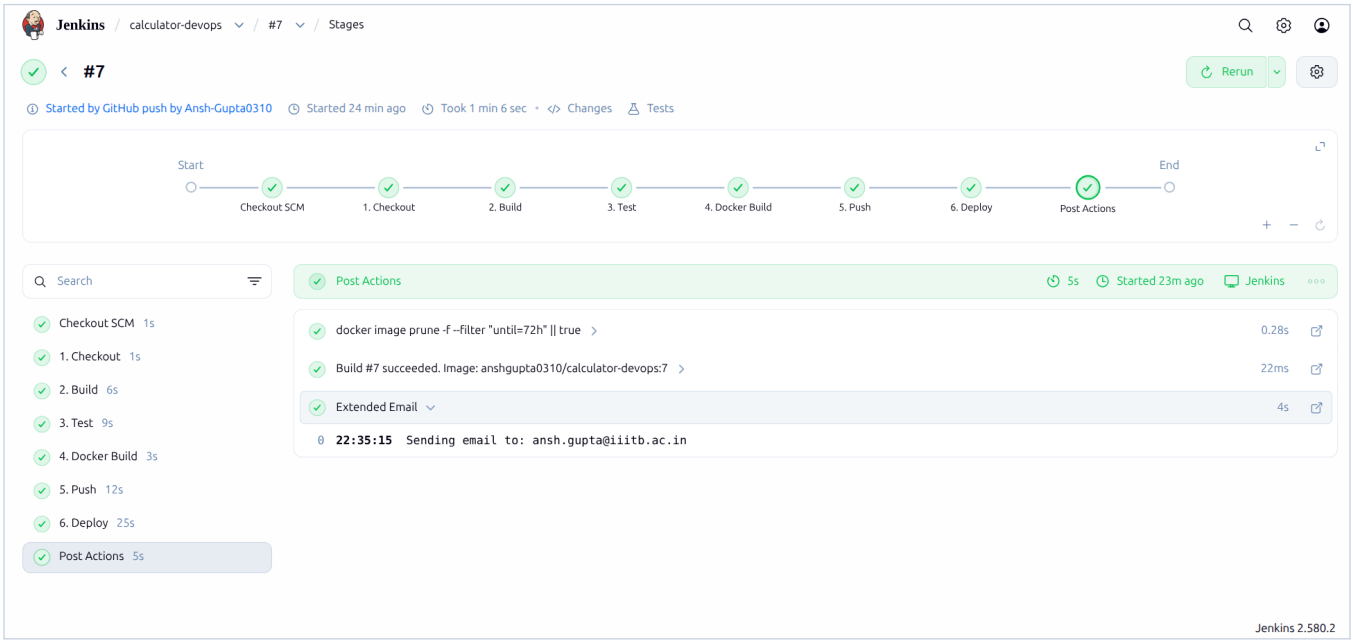


Figure A1. Build #7 stage view: started by a GitHub push, all six stages and post actions green, took 1 min 6 s.

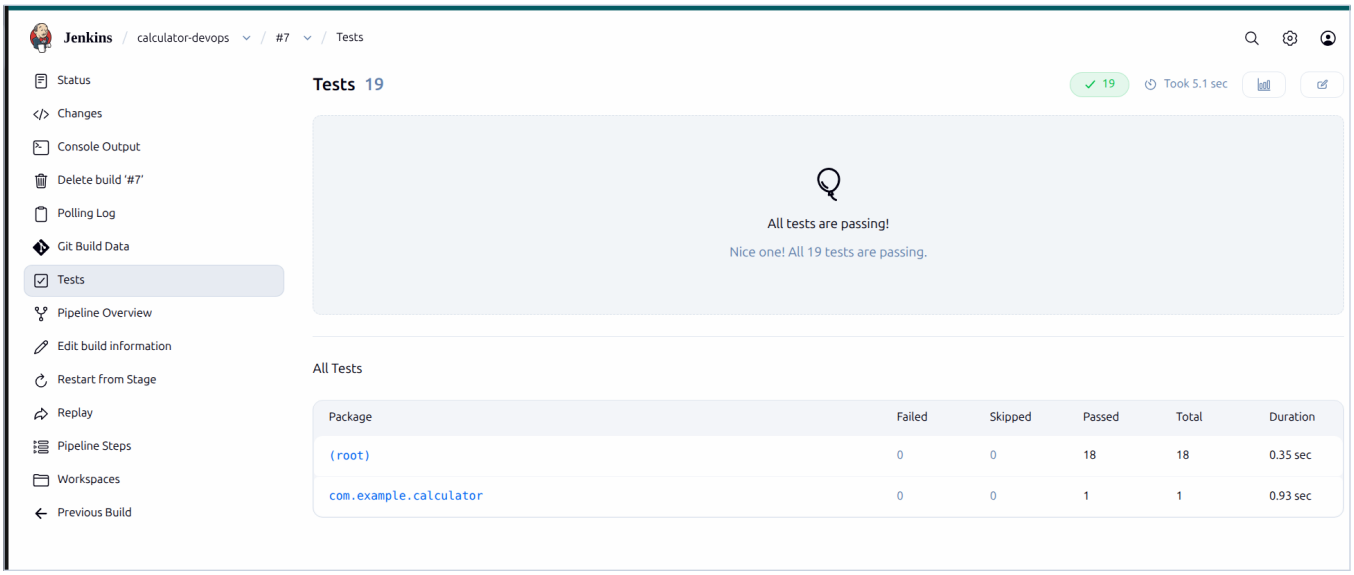


Figure A2. JUnit results published by Jenkins for build #7: 19 tests, 0 failed, 0 skipped.

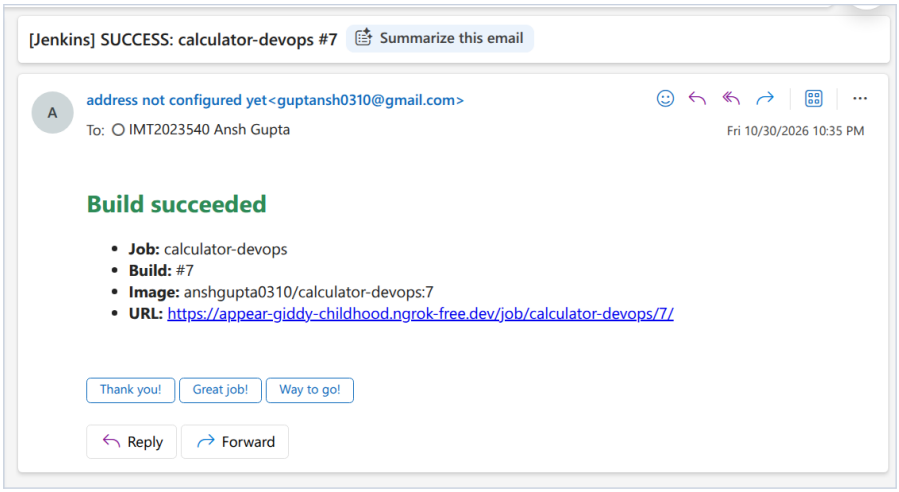


Figure A3. Success email sent by the pipeline post block, with job, build, image tag and link.

```
ansh@ansh:~$ docker ps
CONTAINER ID   IMAGE                                  COMMAND                  CREATED          STATUS                    PORTS
                  NAMES
4e7b5a1b42bd   anshgupta0310/calculator-devops:7   "java -jar /app/app.…"   30 minutes ago   Up 30 minutes (healthy)   8080/tcp
                  calculator
12230fefa2a2   nginx:1.27-alpine                  "/docker-entrypoint.…"   4 hours ago      Up 4 hours                0.0.0.0:80->8
0/tcp, [::]:80->80/tcp   calc-nginx
ansh@ansh:~$
```

Figure A4. docker ps after deployment: calculator (tag :7, healthy) and calc-nginx on port 80.

```
ansh@ansh:~$ curl 'http://localhost/api/v1/add?a=5&b=2.6'

{"operation":"add","a":5.0,"b":2.6,"result":7.6}ansh@ansh:~$ curl 'http://localhost/api/v1/subtract?a=-5&b=2.6'

{"operation":"subtract","a":-5.0,"b":2.6,"result":-7.6}ansh@ansh:~$ curl 'http://localhost/api/v1/multiply?a=-5&b=2.6'

{"operation":"multiply","a":-5.0,"b":2.6,"result":-13.0}ansh@ansh:~$ curl 'http://localhost/api/v1/divide?a=-5&b=2'

{"operation":"divide","a":-5.0,"b":2.0,"result":-2.5}ansh@ansh:~$ curl 'http://localhost/api/v1/divide?a=-5&b=0'
```

Figure A5. Sample API responses through the Nginx reverse proxy on port 80.



Figure A6. Console output: build started by GitHub push, Jenkinsfile obtained from the repository.